
Données Vélib's et introduction à Google BigQuery

Projet NFE204

Boris Cléménçon - 20 février 2015

1. Introduction

L'objectif initial de ce rapport était de collecter des données concernant les vélos en libre service administrés par JCDecaux, de les combiner avec d'autres données pertinentes et de les stocker dans la base de données orientée colonne en ligne de Google : BigQuery. Ayant déjà utilisé BigQuery dans le cadre de NFE212, je pensais réutiliser ce service gratuit pour NFE204. Hélas, entre temps, Google a décidé de rendre payant ses services. Il n'est plus possible d'utiliser BigQuery gratuitement, sauf pour les nouveaux clients. J'ai donc décidé de stocker en local les données récoltées avec MongoDB, puis de présenter BigQuery dans une deuxième partie, et plus généralement des services web de Google.

Ce projet se décompose donc en 2 parties. Dans une première partie, nous développerons l'aspect pratique du projet. Nous présenterons les données et les différentes API utilisées, nous détaillerons le script python permettant de collecter et d'intégrer ces données, puis nous présenterons un script python qui fait des appels à MongoDB afin de générer automatiquement des graphiques. La partie pratique ne traitera donc pas de Google BigQuery.

Dans un deuxième temps, nous présenterons les services web de Google en insistant sur le mode d'indexation de la base de données orientée colonne en ligne BigQuery.

2. Première partie : aspects techniques

2.1. Présentation des données

Les systèmes de vélos en libre service sont nés en France initialement à Lyon puis déployés à grande échelle à Paris en 2007. Aujourd'hui, un nombre important de grandes villes sont équipées d'un tel système. Le concept est toujours la même : un abonné loue un vélo à partir d'une station de départ pour un temps donné et doit le réintroduire dans le système dans une autre station (pouvant être la même station que celle de départ).

En 2012, le système Vélib disposait de 19 000 vélos, enregistrait 110 000 locations quotidiennes et 35 millions sur l'année, soit en moyenne 1,6 vélo loué chaque seconde et de 245.000 abonnés.

JCDecaux met à disposition du public des API REST qui permettent de collecter des données en temps réel sous forme de fichier JSON. Il est ainsi possible de récupérer des données concernant les différents contrats de JCDecaux ainsi que des données concernant toutes les stations de tous les contrats à chaque instant. Par exemple, un objet concernant les contrats est de la forme

```
{
  "country_code": "FR",
  "commercial_name": "cyclic",
  "cities": [
    "Rouen"
  ],
  "name": "Rouen"
}
```

et un objet concernant une station est de la forme

```
{
  "last_update": 1385855802000,
  "available_bikes": 0,
  "available_bike_stands": 25,
  "bike_stands": 25,
  "number": 280,
  "name": "280 HEYSEL / HEISEL",
  "address": "HEYSEL / HEISEL AVENUE DE L'IMPERATRICE CHARLOTTE / KEIZERIN  
CHARLOTTELAAN",
}
```

```
"position": {
  "lng": 4.334831,
  "lat": 50.897522
},
"banking": true,
"bonus": false,
"status": "OPEN",
"contract_name": "Bruxelles-Capitale"
}
```

Cela représentent 3374 stations réparties sur 25 villes (dont 1224 à Paris). Ces données contiennent des données spatiales (la ville, l'adresse et les coordonnées géographiques), chronologique (dernière mise à jour, nombre de vélos disponibles, nombre de places disponibles et status) ou caractéristiques (nom (son identifiant unique), numéro dans le système, nombre de bornes disponibles, présence d'un automate, bonus si on ramène le vélo).

Une application intéressante de cette base de données pourrait être de prédire l'affluence dans les stations ou encore de déclencher automatiquement des interventions de maintenance et de redistribution des vélos. Nous nous proposons de collecter ces données toutes les 5 minutes (période de rafraîchissement du système) et de les intégrer à d'autres données pertinentes : l'altitude des stations et la météo.

La météo est bien évidemment une caractéristique importante. Les API REST de Open Weather Map (OWM) permettent de connaître l'état de la météo à partir d'une position géographique ou du nom d'une ville. Dans une certaine mesure, il est même possible d'avoir accès aux données passées et à des prévisions à trois jours.

```
{
  "cod": 200,
  "name": "Rouen",
  "id": 2982652,
  "coord": {
    "lat": 49.44,
    "lon": 1.09
  },
  "sys": {
    "sunset": 1425231498,
    "sunrise": 1425191853,
    "country": "France",
    "message": 0.0302
  },
  "weather": [
```

```

    {
      "icon": "04n",
      "description": "overcast clouds",
      "main": "Clouds",
      "id": 804
    }
  ],
  "base": "cmc stations",
  "main": {
    "humidity": 95,
    "grnd_level": 1003.37,
    "sea_level": 1017.29,
    "pressure": 1003.37,
    "temp_max": 281.917,
    "temp_min": 281.917,
    "temp": 281.917
  },
  "wind": {
    "deg": 246.005,
    "speed": 8.82
  },
  "clouds": {
    "all": 92
  },
  "dt": 1425245147
}

```

Une autre caractéristique pertinentes est l'altitude. En effet, tout parisien sait bien qu'il est plus probable de trouver une station vide à Montmartre que proche de la Seine. Nous utiliserons donc les API de Google Elevation. Ce service retourne l'altitude à partir de coordonnées géographiques, sous la forme d'un tableau d'objets json de la forme

```

{
  "location": {
    "lng": 4.334831,
    "lat": 50.897522
  },
  "elevation": 64.77242279052734,
  "resolution": 1221.625854492188
}

```

2.2. Script python de chargement des données

Pour collecter ces données en ligne, un script en python a été écrit. Il interroge toutes les 5 minutes successivement les API de JCDecaux, de OpenWeatherMap et de Google Elevation et génère un tableau d'objets json. Ces objets sont chargés en local dans

MongoDB et un fichier compressé est écrit en backup. Nous allons maintenant détailler notre script. Tout d'abord, nous chargeons les packages python qui seront nécessaires pour la suite.

```
import urllib2
import httplib2
import requests
import json
import gzip
import pyowm
import sys
import pyowm.exceptions
import re, urlparse
import numpy as np
from pymongo import MongoClient
from math import pi, sin, cos, acos, exp
from time import time
from time import sleep
from datetime import date, datetime, timedelta
```

Nous définissons ensuite les constantes globales qui permettent de personnaliser le script

```
QUERY_TIME_STEP_IN_MINUTE = 5 # get data every 5 minutes
MAX_NB_ATTEMPTS=10;
GOOGLE_ELEVATION_MAX_LOCATION_PER_REQUEST=52
JCDECAUX_KEY='XXX'
PATH_OUTPUT_REP='./data/'
MONGO_SERVER_ADDRESS='localhost'
MONGO_SERVER_PORT=27017
```

Nous définissons aussi des fonctions utiles pour la suite. Les fonctions suivantes transforment les url avec des caractères spéciaux en url valide

```
def urlEncodeNonAscii(b):
    return re.sub('[\x80-\xFF]', lambda c: '%%02x' % ord(c.group(0)), b)

def iriToUri(iri):
    parts= urlparse.urlparse(iri)
    return urlparse.urlunparse(
        part.encode('idna')
        if parti==1 else urlEncodeNonAscii(part.encode('utf-8'))
        for parti, part in enumerate(parts)
    )
```

Nous nous connectons ensuite au serveur MongoDB, préalablement démarré.

```
try:
    client = MongoClient(MONGO_SERVER_ADDRESS, MONGO_SERVER_PORT)
    db = client.velibs
```

```

collection = db.stations

except Exception as inst:
    print "Exception raised : ", type(inst)
    print "error: " , inst
    exit();

```

Ensuite vient la boucle principale, que appelons toutes les 5 minutes

```

nextUpdate = datetime.now()
nextElevationUpdate = nextUpdate
nextWeatherUpdate = nextUpdate

# Request periodically
elevations=[]
weatherData=[]
while True:

    now = datetime.now()
    # Try regularly if we passed the next time for update
    if now<nextUpdate:
        sleep(QUERY_TIME_STEP_IN_MINUTE*60/10)
        continue

    # If so, update nextupdate time and query
    nextUpdate = nextUpdate+timedelta(minutes=QUERY_TIME_STEP_IN_MINUTE)

    # New records
    print "*****" + datetime.strftime(now, '%Y-%m-%d %H:
%M:%S') + "*****"

```

Nous commençons par charger les données JCDecaux concernant les différents contrats avec une agglomération. Ces données ne changent a priori pas souvent, mais leur volume étant modeste, nous les interrogeons toutes les cinq minutes.

```

url='https://api.jcdecaux.com/vls/v1/contracts?apiKey=' + JCDECAUX_KEY
contracts=None
for i in range(MAX_NB_ATTEMPTS):
    try:
        content = urllib2.urlopen(url)
        contracts = json.loads(content.read())
        print "> %s..." % url[0:50]
        break
    except:
        print "[WAR] getBikes: Could not open %s" % url

if i>=MAX_NB_ATTEMPTS:
    print "[ERR] Last attempt to connect to JCDecaux failed. »"

```

Puis nous chargeons les données JCDecaux concernant les stations. La plupart de ces données sont susceptibles de changer toutes les cinq minutes.

```
url='https://api.jcdecaux.com/vls/v1/stations?apiKey=' + JCDECAUX_KEY
stations=None
for i in range(MAX_NB_ATTEMPTS):
    try:
        content = urllib2.urlopen(url)
        stations = json.loads(content.read())
        print "> %s..." % url[0:50]
        break
    except:
        print "[WAR] getBikes: Could not open %s" % url

if i>=MAX_NB_ATTEMPTS:
    print "[ERR] Last attempt to connect to JCDecaux failed."
```

Ensuite, nous souhaitons connaître l'altitude de ces stations. Pour cela nous faisons appel aux API Google Elevation. Pour minimiser le nombre de requêtes, nous regroupons d'abord toutes les coordonnées géographiques de toutes les stations. Ensuite, par tranche de 50 environ, nous interrogeons les API afin de minimiser le nombre de requêtes. Nous renouvelons cette opération une fois par jour. Bien sûr, à moins d'une activité sismique particulièrement intense (et en quel cas, l'intégrité des vélib's sera la moindre préoccupation des autorités), l'altitude n'est pas susceptible de changer tous les jours. En revanche, de nouvelles stations apparaissent régulièrement et il est préférable de mettre à jours ces données régulièrement afin de ne pas omettre de nouvelles stations.

```
if now>=nextElevationUpdate:
    nextElevationUpdate = nextElevationUpdate+timedelta(hours=24)

print '> Query Google Elevation API'
for station in stations:
    position=station['position']
    lng=float(position['lng'])
    lat=float(position['lat'])
    elevations.append({'lng':lng,
                       'lat':lat,
                       'number':station['number'],
                       'contract_name':station['contract_name']
                      })

try:
    n=len(elevations)
    K=n/GOOGLE_ELEVATION_MAX_LOCATION_PER_REQUEST
    for i in range(0,K-1):
        a=GOOGLE_ELEVATION_MAX_LOCATION_PER_REQUEST*i
        b=min(n,a+GOOGLE_ELEVATION_MAX_LOCATION_PER_REQUEST)-1
```

```

locstr = ''
for coord in elevations[a:b]:
    if coord!=elevations[a]: locstr += '|'
    locstr += '%f,%f' % (coord['lat'],coord['lng'],)

url='https://maps.googleapis.com/maps/api/elevation/json?
locations=' + locstr
#print '> %s... [%d, %d]' % (url[0:100],a,b)
content = urllib2.urlopen(url)
results = json.loads(content.read())

# check error
if results['status'] == 'OVER_QUERY_LIMIT':
    print '[ERR] ' + results['error_message']
    break

m = len(results['results'])
for j in range(a,b):
    elevations[j]['elevation']=results['results'][j-a]['elevation']

except Exception as inst:
    print "Exception raised : ", type(inst)
    print "error: " , inst
    print url

```

Ensuite, nous collectons toutes les villes dans lesquelles JCDecaux gère les vélos publics et nous évaluons la météo toutes les heures. Nous gardons en particulier la température, l'humidité, la pression atmosphérique, la vitesse du vent et l'heure de lever et du coucher du soleil.

```

if now>=nextWeatherUpdate:
    nextWeatherUpdate = nextWeatherUpdate+timedelta(hours=1)

locations=[]
for contract in contracts:
    city = contract['name'];
    city = city.lower()
    locations.append({'country_code':contract['country_code'],
'city':city})

print '> Query Open WeatherMap API'
weatherData=[]
for location in locations:
    iri='http://api.openweathermap.org/data/2.5/weather?
q="'+location['city']+'",'+location['country_code']
    url=iriToUri(iri)

    for i in range(MAX_NB_ATTEMPTS):
        try:
            content = urllib2.urlopen(url)
            result = json.loads(content.read())

```

```

weather={'temp':0,'humidity':0,'pressure':0,'wind':0,'sunrise':
0,'sunset':0}
weather['temp']=result['main']['temp']
weather['humidity']=result['main']['humidity']
weather['pressure']=result['main']['pressure']
weather['wind']=result['wind']['speed']
weather['sunrise']=result['sys']['sunrise']
weather['sunset']=result['sys']['sunset']
weatherData.append({'location':location, 'weather':weather})
    break;

except Exception as inst:
    print "[WAR] Could not get weather for URL %s" % url
    print "Exception raised : ", type(inst)
    print "error: " , inst
    print url

if i>=MAX_NB_ATTEMPTS:
    print "[ERR] Last attempt to connect to JCDecaux failed."

```

Enfin, nous intégrons toutes les données précédemment collectées dans un seul et même objet par station.

```

for station in stations:
    contract = station['contract_name']
    number=station['number']

    # Parse the list of city weather
    for w in weatherData:
        if w!=None and w['location']['city']==contract.lower():
            station['weather']=w['weather']
            break

    # Parse the list of elevations
    for z in elevations:
        if z['number']==number and z['contract_name']==contract and
'elevation' in z.keys() :
            station['elevation']=z['elevation']

```

Nous écrivons ensuite un fichier compressé. Initialement, ce fichier devait être stocké sur Google Storage afin d'être chargé dans BigQuery. Cette étape nous sert dorénavant uniquement de sauvegarde.

```

# Write gzip file for static data (once a day)
try:
    filename = PATH_OUTPUT_REP+datetime.strftime(now,'%Y%m%d%H%M%S')
+'.json.gz'
    f = gzip.open(filename, 'w')
    for station in stations:
        f.writelines(json.dumps(station)+'\n')
    f.close()
    print '> write new file ' + filename

```

```
except Exception as inst:
    print "Exception raised : ", type(inst)
    print "error: " , inst
```

Enfin, nous écrivons nos objets correspondant à une station dans MongoDB.

```
# Store in MongoDB
for station in stations:
    try:
        collection.insert(station)

    except Exception as inst:
        print "Exception raised : ", type(inst)
        print "error: " , inst
print '> write objects in MongoDB'
```

Et nous nous déconnectons de MongoDB

```
# disconnect to mongodb
client.close()
```

2.3. Script python de chargement des données

Le script précédent peut être lancé en background. Toutes les cinq minutes, les données concernant les stations sont alors mises à jour et stockées dans MongoDB. Il serait dommage de se quitter sans avoir utilisé ces données!

Le script suivant génère un graphique mettant en relation le taux d'occupation moyen des stations (défini comme le nombre de vélo disponible divisé par le nombre total de bornes) en fonction de l'altitude à l'aide du package matplotlib.

```
import json
import sys
import numpy as np
from pymongo import MongoClient
from math import pi, sin, cos, acos, exp
import numpy as np
import matplotlib.pyplot as plt
```

Nous définissons ensuite les variables globales utiles.

```
MONGO_SERVER_ADDRESS='localhost'
MONGO_SERVER_PORT=27017
MONGO_DB='velibs'
MONGO_COLLECTION='stations'
```

Nous connectons dans ce script MongoDB.

```
try:
    client = MongoClient(MONGO_SERVER_ADDRESS,MONGO_SERVER_PORT)
    db = client.velibs
    collection = db.stations

except Exception as inst:
    print "Exception raised : ", type(inst)
    print "error: " , inst
    exit();
```

Nous écrivons une requête permettant de sélectionner les objets qui nous intéressent. Nous nous limiterons aux stations parisiennes. Dans ces requêtes nous faisons une projection sur les données qui nous intéressent, à savoir l'élévation, le nombre de vélos disponibles et le nombre total de bornes

```
# Get the number of available bike stands in every stations in Paris
query = {'contract_name':'Paris'}
projection={'number':1,'bike_stands':1,'available_bikes':1,'elevation':1}

y={}
x={}
cpt={};
try:
    stations = collection.find(query,projection)
    for station in stations:
        occupation=float(station['available_bikes'])/
float(station['bike_stands'])
        if 'elevation' in station.keys():
            key=station['number']
            if key not in y.keys():
                x[key]=station['elevation']
                y[key]=occupation
                cpt[key]=1
            else:
                cpt[key]+=1
                y[key]+=occupation
        else :
            print 'Missing elevation for:'
            print station

except Exception as inst:
    print "Exception raised : ", type(inst)
    print "error: " , inst
```

Nous créons enfin le graphique final

```
# the scatter plot of the data
u=[ ]
```

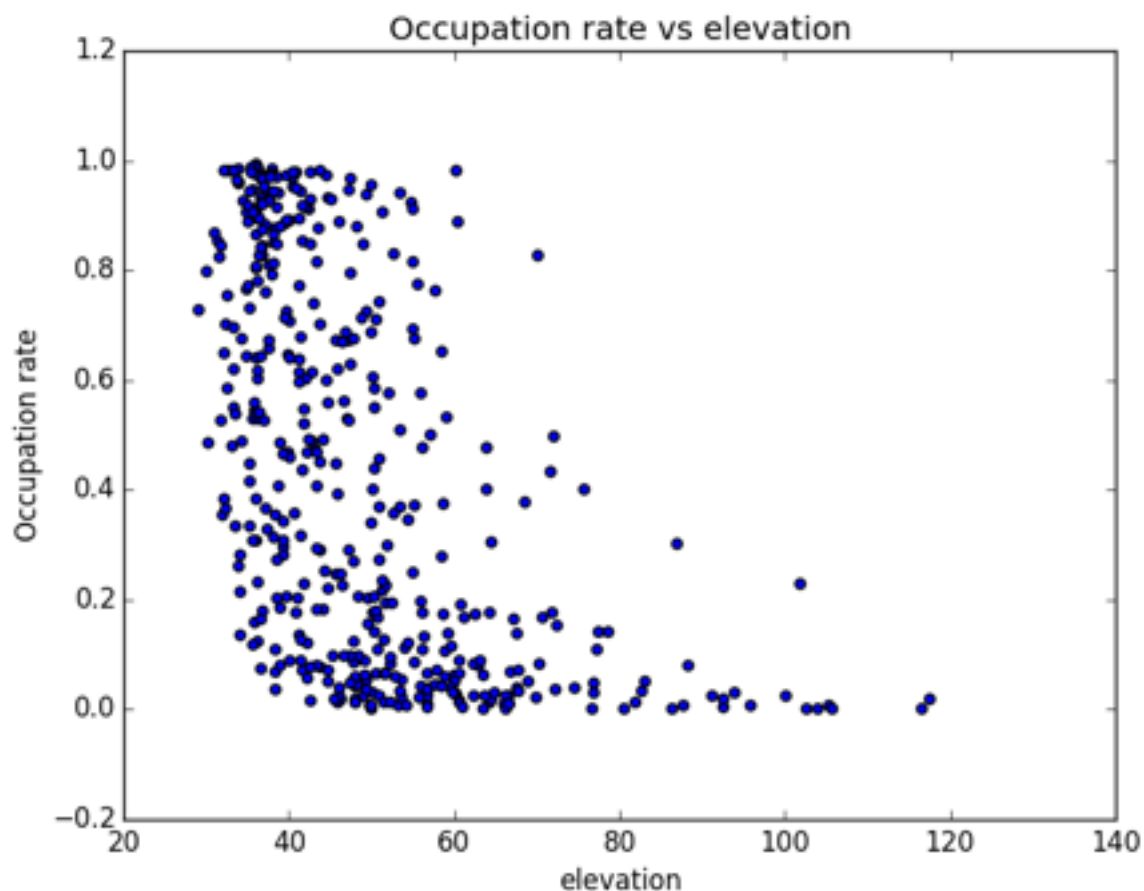
```
z=[]
keys = y.keys()
for key in keys:
    u.append(x[key])
    z.append(y[key]/cpt[key])

plt.scatter(u,z)
plt.title('Occupation rate vs elevation')
plt.xlabel('elevation')
plt.ylabel('Occupation rate')
plt.show()
```

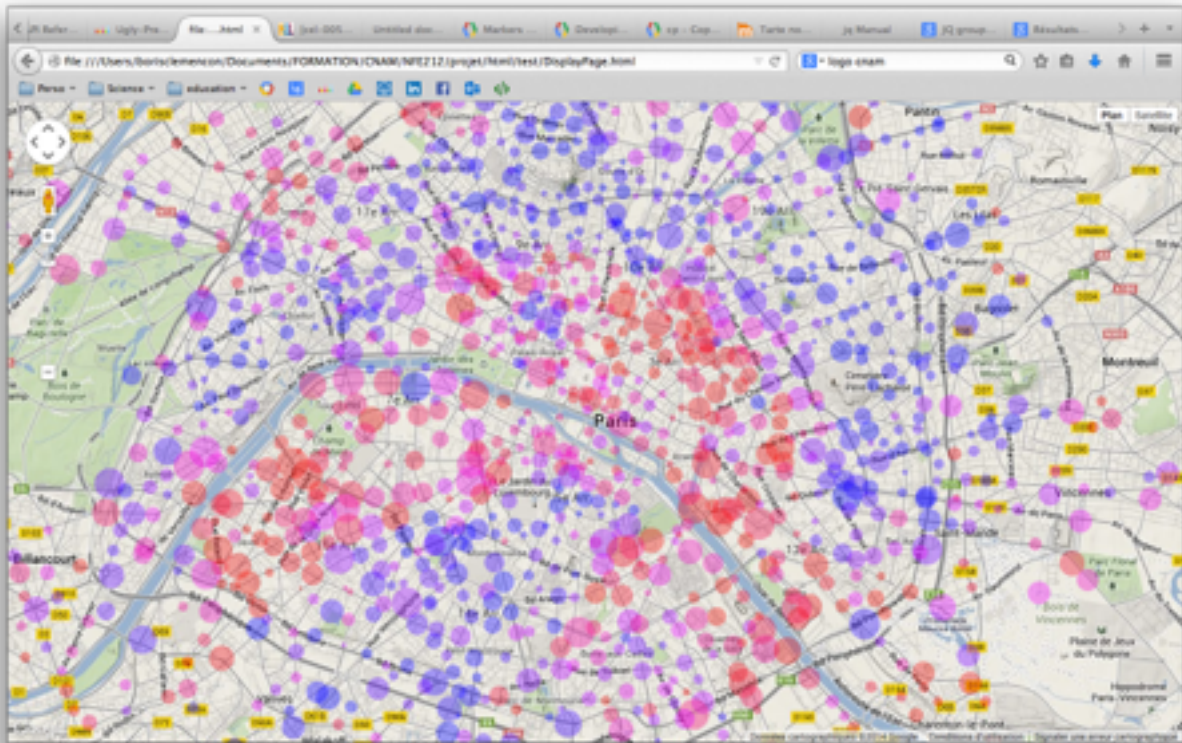
Enfin, nous fermons les connexions MongoDB

```
# disconnect to mongodb
client.close()
```

Le résultat de ce script est le scatter plot suivant. Une brève interprétation consiste à dire que la variance des stations en base altitude est importante, alors qu'elle est faible et que la moyenne est basse pour les stations en plus haute altitude.



Une autre application qui a été faite dans le cadre de l'UE NFE212 est une carte interactive sur laquelle les stations de vélib's apparaissent selon une couleur différente en fonction du taux d'occupation à un instant donné. Cette carte a été réalisée avec les API Javascript de Google Map, et les données concernent les vélib's pendant un mois. Cette carte n'utilise pas BigQuery, bien que les données y aient été stockées. Il s'agit en fait d'un simple fichier JSON contenu sur Google Storage et qui est appelé à chaque fois que l'on met la carte à jour (très peu efficace!).

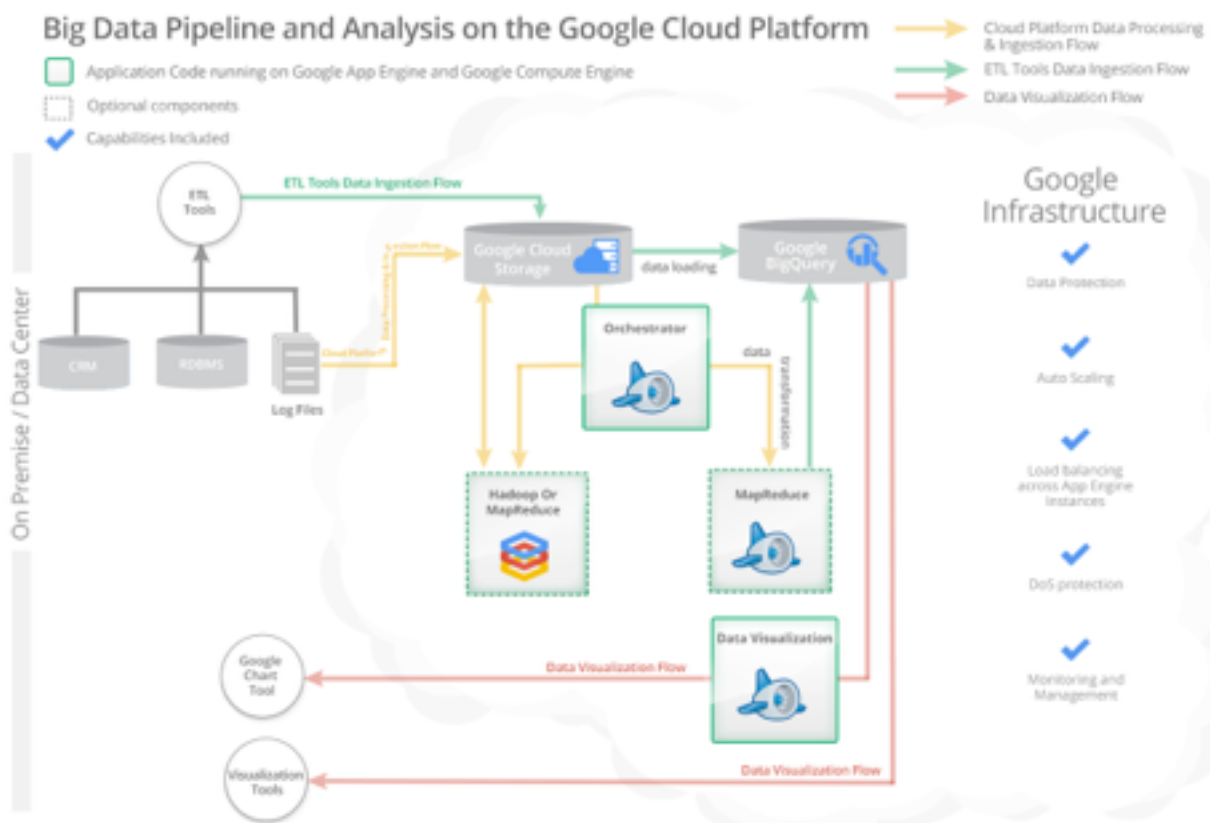


L'application idéale serait de faire tourner le script python en background sur Google App Engine et de stocker les données collectées dans BigQuery. Une application client se connecterait à cette base de données, extrairait les données dont elle a besoin dans un dialecte de SQL et afficherait le résultat sur une carte interactive en utilisant les API de Google MAP. Des calculs plus sophistiqués devraient permettre de sortir des KPI intéressants permettant de décider une intervention de maintenance ou de redistribution des vélos (voir projet STA112).

3. Google BigQuery

3.1. Présentation

Google web services est un ensemble de services en lignes, d'API, de base de données, de solutions de stockage, et d'exécution IaaS (Infrastructure as a Service). L'idée de Google Web Services est de stocker les données et d'exécuter des scripts dans le cloud afin que des applications, par exemple mobiles, puissent alimenter, partager, et exploiter de grands volumes de données de manière scalable. La figure ci dessous représente les différents processus de traitement de données autorisés par Google Web Service¹.



¹ <https://cloud.google.com/developers/articles/getting-started-with-google-bigquery/>

Google BigQuery est le service de base de données en ligne de Google Web Services, reposant sur la technologie *Dremel*. C'est cette technologie qui est à la base du projet Apache Drill. Ce système de base de données orientée colonnes, comparable aux frameworks Apache *HBase* et *Cassandra*, est capable de gérer de très grands volumes de données.

Par exemple, dans l'idéal, notre script de collecte des données devrait tourner en background sur Google App Engine. Il collecterait les données comme précédemment mais stockerait les données non plus en local dans MongoDB, mais dans le cloud, dans la base de données orientée colonne BigQuery (directement ou en stockant des fichiers intermédiaires dans Google Storage). Des solutions de cache permette d'accéder plus rapidement aux données les plus utilisées afin de gagner du temps.

3.2. Sous le capot: Dremel

BigQuery repose sur la technologie de base de données distribuées Dremel, déjà utilisée intensivement par Google. L'architecture parallèle basée sur les arbres d'exécution multi-niveaux² permettent de distribuer massivement les données et les calculs, et ainsi de garantir de bonnes performances pour des jeux de données de plusieurs teraoctets.

Les données se répartissent dans des tables, elles-mêmes regroupées en datasets où sont définis les droits d'accès. Il est possible d'ajouter de nouvelles données, mais pas de les supprimer ou de les modifier, si ce n'est en effaçant la table entièrement. Le schéma des tables est évolutif. Il est ainsi possible d'insérer des données ne correspondant pas exactement aux schéma d'origine.

Comme la plupart des systèmes de base de données orientée colonne, il existe des similitude avec les systèmes relationnels classiques. En particulier, le langage de requête est très proche du SQL traditionnel. Ceci est un grand avantage et permet de créer de nouvelles applications plus sagement qu'avec des bases de données documentaires comme MongoDB.

² http://static.googleusercontent.com/external_content/untrusted_dlcp/research.google.com/en/us/pubs/archive/36632.pdf

Les requêtes s'écrivent dans un dialecte de SQL à partir d'une interface web dédiée, de lignes de commande (utilitaire bq) ou d'applications JAVA, PHP ou Python via des API REST. Il est donc possible d'accéder aux données en local via bq, et via des applications sophistiquées hébergées dans App Engine ou ailleurs.

La différence repose dans la manière dont les données sont stockées. Contrairement au SGBD relationnels, les données ne sont pas stocker sous forme de lignes, mais de colonnes. Cette structure permet des schémas moins rigides avec des colonnes imbriquées, ce qui permet une certaine souplesse dans la définition initiale de l'architecture. L'article *Dremel: Interactive Analysis of Web-Scale Datasets* de Melnik & al³ décrit en détail l'architecture par colonne de Dremel.

Notons enfin que. la base de données étant en ligne, il est important d'assurer l'intégrité et la confidentialité des données. Le système d'authentification OAuth permet d'assurer la sécurité et l'intégrité des données entre une application client et le serveur.

3.3. Interaction avec la base de données

Par exemple, nous allons accéder à la base de données public *samples.shakespeare*. La commande suivante retourne le schéma de la table (les exemples suivant sont ici de la page web <https://cloud.google.com/bigquery/bq-command-line-tool-quickstart>) .

```
$ bq show publicdata:samples.shakespeare
tableId      Last modified      Schema
-----
shakespeare  01 Sep 13:46:28    | - word: string (required)
                  | - word_count: integer (required)
                  | - corpus: string (required)
                  | - corpus_date: integer (required)
```

Pour exécuter une requête, nous utilisons la commande suivante (remarquons la syntaxe de la requête très proche du SQL).

```
$ bq query "SELECT word, COUNT(word) as count FROM
publicdata:samples.shakespeare WHERE word CONTAINS 'raisin' GROUP BY word"
```

³ http://static.googleusercontent.com/external_content/untrusted_dlcp/research.google.com/en/us/pubs/archive/36632.pdf

```
Waiting on job_dcda37c0bbbed4c669b04dfd567859b90 ... (0s) Current status: DONE
```

word	count
Praising	4
raising	5
raisins	1
praising	7
dispraising	2
dispraisingly	1

Voici à titre d'exemple une autre requête

```
$ bq query "SELECT name,count FROM mydataset.names2010 WHERE gender = 'M' ORDER BY count ASC LIMIT 5"
```

```
Waiting on job_556ba2e5aad340a7b2818c3e3280b7a3 ... (1s) Current status: DONE
```

name	COUNT
Aarian	5
Aaidan	5
Aamarion	5
Aadhavan	5
Aaqib	5

Il est possible d'importer des données dans la table à partir de fichier compressés stocker sur Google Storage. Remarquons que notre script écrit de tels fichiers.

Il est aussi possible d'écrire des application, en python par exemple, interagissant avec cette base. Le script suivant montre comment s'authentifier et comment se connecter à BigQuery.

```
import httplib2

from apiclient.discovery import build

from oauth2client.client import flow_from_clientsecrets
from oauth2client.file import Storage
from oauth2client import tools

# Enter your Google Developer Project number
PROJECT_NUMBER = '12345XXXXXXX'

FLOW = flow_from_clientsecrets('client_secrets.json',
                              scope='https://www.googleapis.com/auth/
bigquery')
```

```
storage = Storage('bigquery_credentials.dat')
credentials = storage.get()

if credentials is None or credentials.invalid:
    # Run oauth2 flow with default arguments.
    credentials = tools.run_flow(FLOW, storage,
tools.argparser.parse_args([]))

http = httplib2.Http()
http = credentials.authorize(http)

bigquery_service = build('bigquery', 'v2', http=http)
```

Pour envoyer une requête, la syntaxe est la suivante.

```
# Create a query statement and query request object
query_data = {'query': 'SELECT TOP(title, 10) as title, COUNT(*) as
revision_count FROM [publicdata:samples.wikipedia] WHERE wp_namespace =
0;'}
query_request = bigquery_service.jobs()

# Make a call to the BigQuery API
query_response = query_request.query(projectId=PROJECT_NUMBER,
body=query_data).execute()
```